UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning for NLP

Student name: *Vasilakis Nikolaos*
sdi: <sdi2200016>

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Spring Semester 2025*

Contents

1 Abstract **2**

2 Data processing and analysis **2**
2.1 Pre-processing . . . 2
2.2 Analysis . . . 3
2.3 Data partitioning for train, test and validation . . . 5
2.4 Vectorization . . . 5

3 Algorithms and Experiments **5**
3.1 Experiments . . . 5
3.1.1 Table of trials . . . 11
3.2 Hyper-parameter tuning . . . 12
3.3 Optimization techniques . . . 12
3.4 Evaluation . . . 12
3.4.1 ROC curve . . . 13
3.4.2 Learning Curve . . . 14
3.4.3 Confusion matrix . . . 14

4 Results and Overall Analysis **15**
4.1 Results Analysis . . . 15
4.1.1 Best trial . . . 16
4.2 Comparison with the first project . . . 16

5 Bibliography **16**

1. Abstract

This is a report following the attempt to build a sentiment classifier on a given English Twitter data set. The classifier would assign tweets with a sentiment, with 1 representing a positive sentiment and 0 representing a negative sentiment. The sentiment classifier would be developed using a deep neural network, while the inputs of our model are Word2Vec embeddings.

Moreover, each tweet in our database contained an ID, Text and a Label. The ID was used to uniquely identify the tweet, the Text was the Tweet's content, and the Label was indicating whether a tweet had either a positive or a negative sentiment.

2. Data processing and analysis

2.1. Pre-processing

Realizing that the tweets we had contained a lot of information, the idea of preprocessing the data with the aim of cleaning up noise was necessary. There are 6 techniques that are used to clean the data properly.

- 1. Converting all text to lowercase: This is done because the meaning of the word will be the same regardless of how many capital letters there.
- 2. Removing URLs and mentions (@s): URLs and mentions essentially give no information about the sentiment of a sentence.
- 3. Removing punctuation and non-ASCII characters: Punctuation is removed, apart from exclamation marks (!) and question marks (?) because these two can give information about the sentiment of a sentence. Fullstops, commas, etc. do not give any information about the sentiment of a sentence; therefore, they are removed. Moreover, non-ASCII characters are also removed because they do not contain relative information about a sentence and its sentiment.
- 4. Removing hash tags: Hashtags can indicate if a tweet has a positive or a negative sentiment, however, they are removed for a number of reasons. Firstly, hash tags encode words in an unstructured way (for example, they contain compound words), therefore, our model is likely to not understand the true meaning of the hash tag. Also, hash tags might cause overfitting since they the model treats hash tags as unique features and words, therefore making it much harder to generalize.
- 5. Correction of spelling mistakes: Common spelling mistakes as well as slang is corrected. For instance, the word "gud" is converted to "good", the word "wanna" is converted to "want to", etc. This is done in order to clean up some of the noise that is caused by the data.
- 6. Converting many characters into one: The final technique used, is converting more than 3 characters into a single one. For example, the word "heyyyyy" in a tweet is converted to the word "hey". This is also done in order to clean up noisy data and have a better model.

- 7. Stopword removal: Common English language words such as 'am', 'or', 'and' are removed. Moreover, common words used both in positive and negative tweets are removed since they give no information on whether a tweet has a positive or a negative sentiment. The common words used in both positive and negative tweets were found through a visualization of data analysis in positive and negative tweets seperately (see section 2.2. Analysis)

2.2. Analysis

Along with the data preprocessing, an data analysis is also being done, in order to understand and vizualise the data in a better way and understand their nature.

The first data analysis that must be done, is to find out whether the data are balanced or not. A balanced data set makes the training of the model easier, because the model will not have any bias towards a particular label. Consequently, the positive and negative instances must be roughly the same.

After the analysis, it is clear that the data set is balanced with 74196 positive instances and 74192 negative instances.

Figure 1: Class distribution in the training data set

Data analysis is also used for the most common words in the unpreprocessed data. The most common words are separated between positive and negative instances. The reason this analysis is done is because it allows for better stopword removal as well as better filtering of overly common words that exist in both positive and negative

instances. In short, it is very useful in order to remove words that exist in both instances and do not help us separate positive and negative tweets.

Below is the visualization of the most frequent words in positive and negative instances separately.

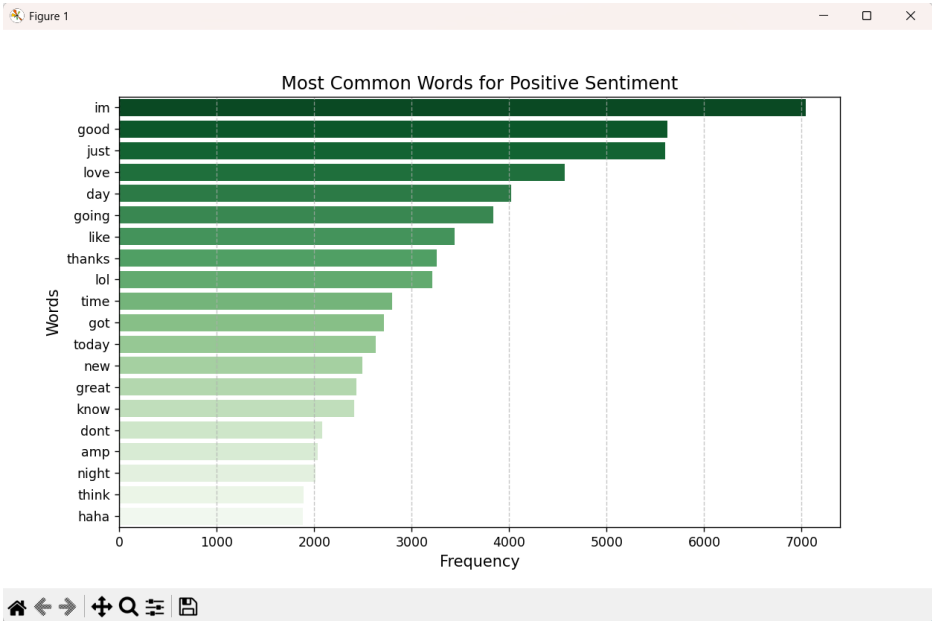


Figure 2: Most Common Words for Positive Sentiment

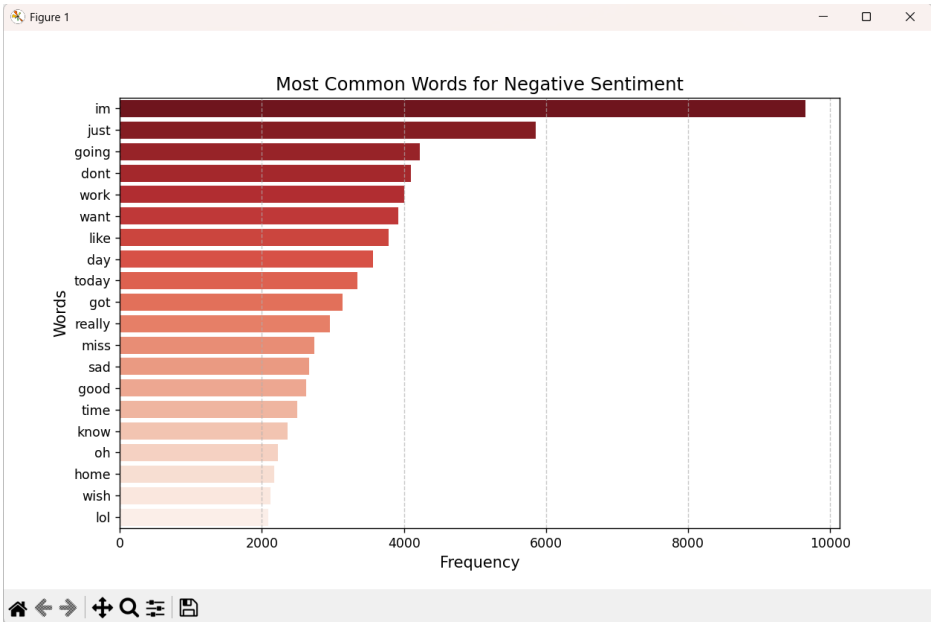


Figure 3: Most Common Words for Negative Sentiment

It is clear that there are words ("i'm", "just", "going", "don't", "day", "got", "like", "today") which appear frequently in positive and negative instances as well.

This is benefitial in order to understand which words might need to be removed later on, since they offer no information on whether a sentence/tweet has a positive or a negative sentiment.

Note: It is important to mention that the exploratory data analysis was done only on the training data. This is because we aim to avoid any potential influence from the test data, as the test set is meant for evaluating the final model's performance. Performing EDA on the test data could inadvertently lead to biases that would affect the model's generalization ability. For the same reason, no preprocessing was performed on the validation data, as it is used for hyperparameter tuning. Analyzing the validation set could have biased the selection of hyperparameters and skewed the results, leading to overfitting.

2.3. Data partitioning for train, test and validation

The data set was already partitioned when it was given to us. The size of the training and validation dataset was already well split (approximately 75% for training data set and 25% for validation data set), so there was no re-arrangement. The training data set was used to train our model and later the model was tuned based on the results given from the validation data set. No tuning or optimization was performed on the basis of the test data set. The test data set was only used for the Kaggle competition and in order to see how good is the generalization of the model.

2.4. Vectorization

A pre-trained Word2Vec model was used to generate word embeddings. Specifically among several options such as Google-news-300, GloVe-6b-50d', the model which was chosen was 'Glove-Twitter-200'. This choice was made because the embeddings were trained on a twitter data set, making it the best possible fit for our task. Moreover, the 200-dimensional vector was a balanced trade-off between training cost and contextual accuracy.

3. Algorithms and Experiments

3.1. Experiments

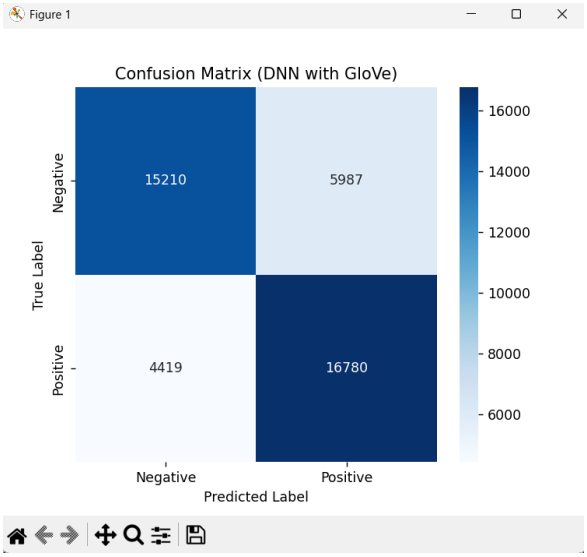
In order to evaluate the accuracy of the classifier the training data set was used to train our model, and then the validation data set to fine tune and optimize our model. The test data set was only used to calculate the final score in the Kaggle competition and see how well the model generalized.

In order to build an efficient neural network, the initial approach that was used was constructing the simplest neural network possible and gradually increasing its complexity. The initial model consisted of only two hidden layers, with no activation functions, batch normalization, or other optimization techniques. A batch size of 64, the AdamW optimizer, and the BCELoss() function were used, with a learning rate of 1e-4, the same as suggested in the tutorial. At this stage, the hyperparameters (such as the optimizer and learning rate) were chosen somewhat randomly, with the intention of tuning them later.

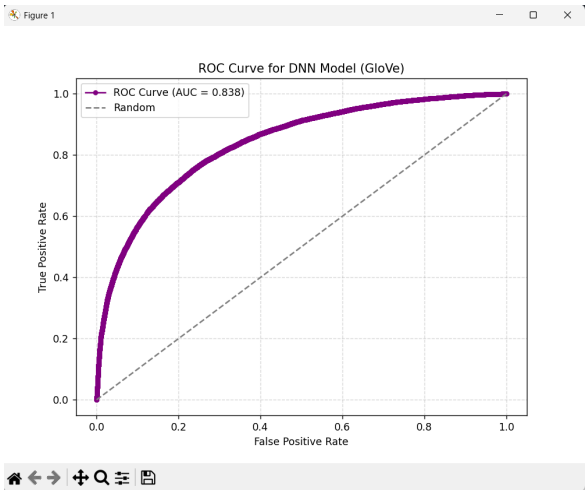
The results on such a simple neural network were underwhelming as expected, with accuracy being low. Through experiments with the number of neurons in the hidden layers, the value of 512 neurons in the first hidden layer and 256 in the second

hidden layer was chosen as the one with the best accuracy. Accuracy of the initial neural network was 0.75455

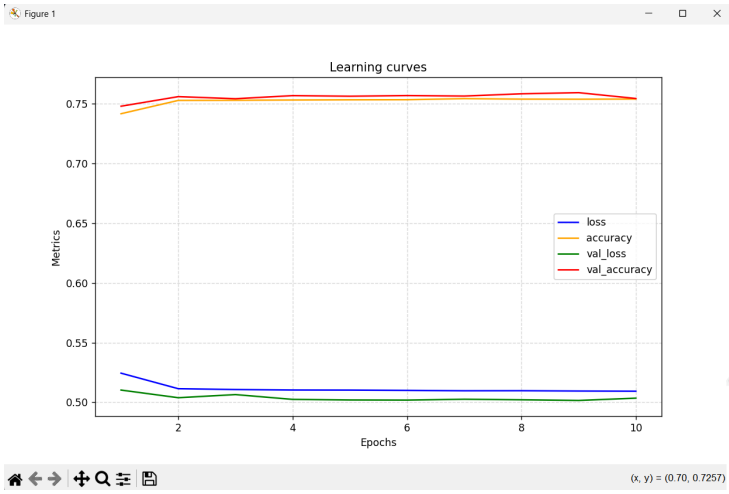
Below are the confusion matrix, the ROC curve and the learning curve for the neural network with two hidden layers:



(a) Confusion Matrix



(b) ROC Curve

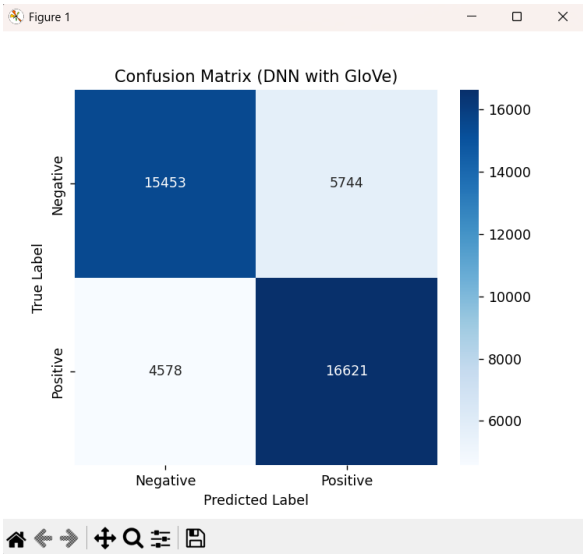


(c) Learning Curve

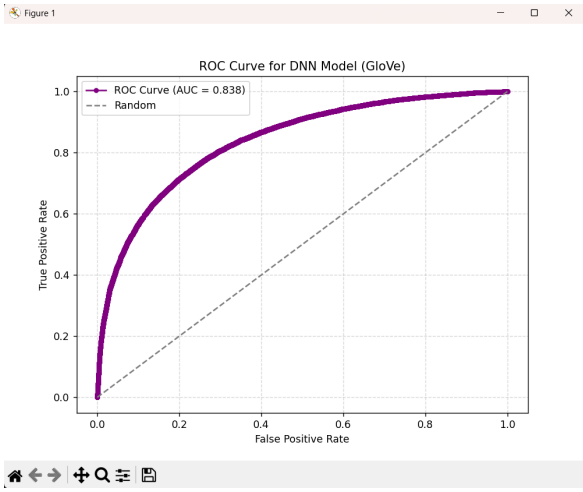
Figure 4: Performance of the initial neural network with two hidden layers

Afterwards, more hidden layers were added to the architecture of the neural network. Initially, a third hidden layer was added. The best performance of a neural network with three hidden layers had 128 or 64 neurons in the third hidden layer. Ultimately, 64 neurons were chosen for the third hidden layer because of lower computational complexity. The accuracy of the neural network with three hidden layers was increased from 0.75455 to 0.75653. Later, a fourth hidden layer was added. Experiments were made with different values for the fourth hidden layer but there wasn't enough accuracy increase to warrant adding a fourth hidden layer and also the computation complexity was higher.

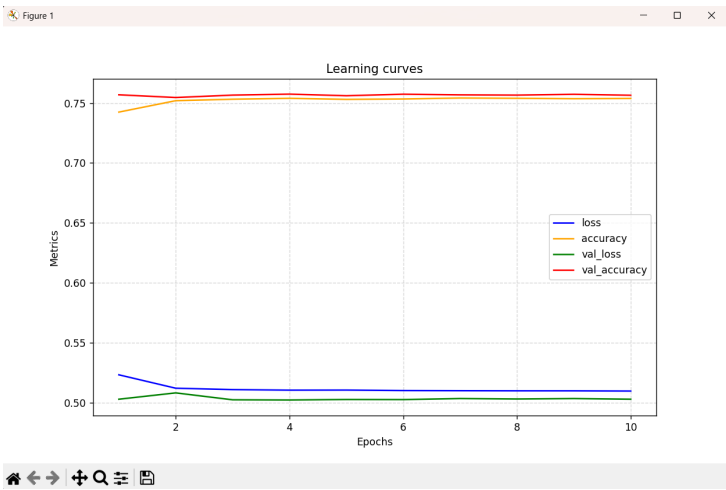
Below are the confusion matrix, the ROC curve and the learning curve for the neural network with three hidden layers:



(a) Confusion Matrix



(b) ROC Curve



(c) Learning Curve

Figure 5: Performance of the neural network with three hidden layers

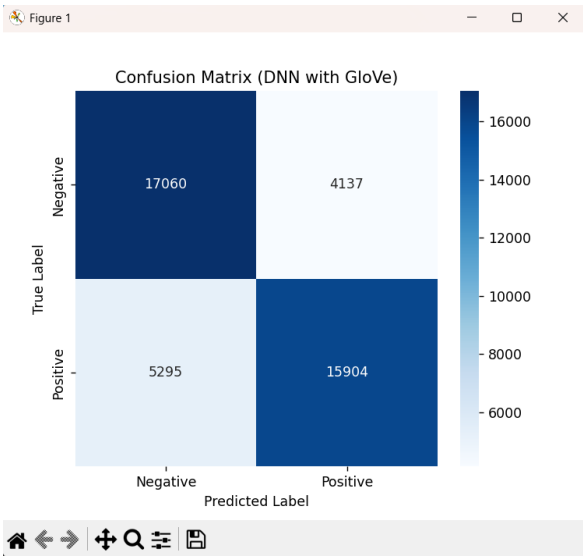
Continuing with the construction of a more complex neural networks, activation functions were later used. The most significant improvent came when using activation functions on our model. Adding the ReLU activation function - even with no weight initialization - saw an increase on our model's accuracy by almost 2%, going from 0.75653 to 0.77526.

In fact, multiple other activation functions such as Leaky ReLU, SELU and GELU were used with decrease in accuracy. Furthermore, when weight initialization was also used, no increase was noticed.

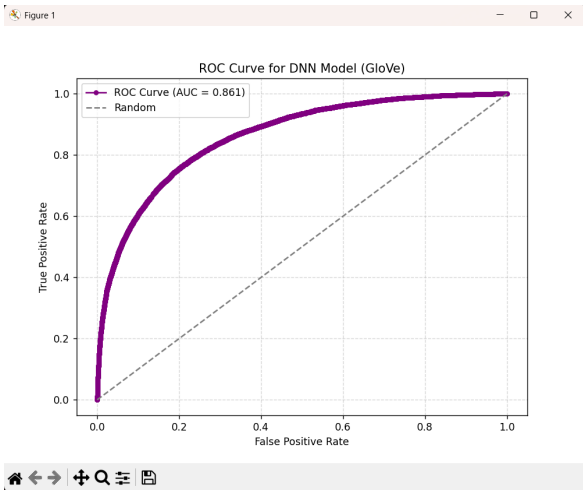
In conclusion, the simplest activation function (ReLU), was the one that worked better. This initially came as a surprise, but since building a linear classifier for a Twitter data set is a relatively simple task, more complex activation functions could have increased the complexity on a problem that isn't much complex to begin with.

However, one major drawback is the learning curves generated from our model. It is clear in the graph of the learning curve below, that there is low overfitting and also limited generalization of the model

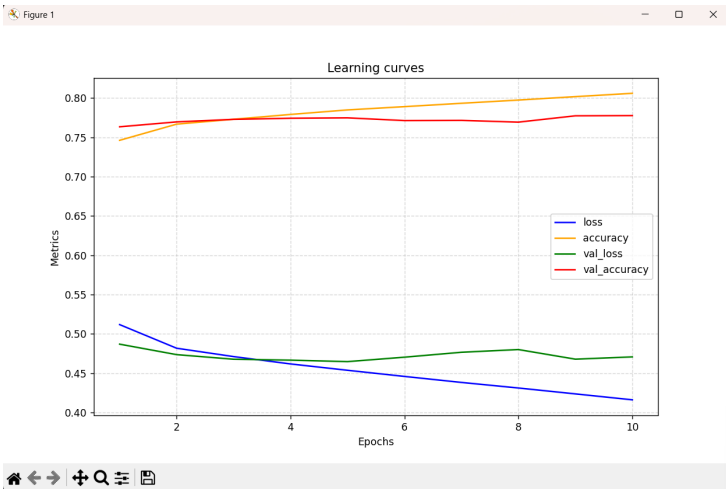
Below are the confusion matrix, the ROC curve and the learning curve for the neural network with three hidden layers and ReLU activation function:



(a) Confusion Matrix



(b) ROC Curve



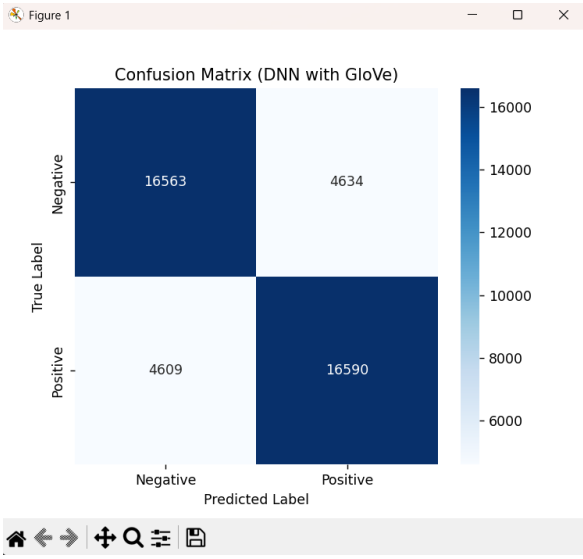
(c) Learning Curve

Figure 6: Performance of the neural network with three hidden layers and ReLU activation function

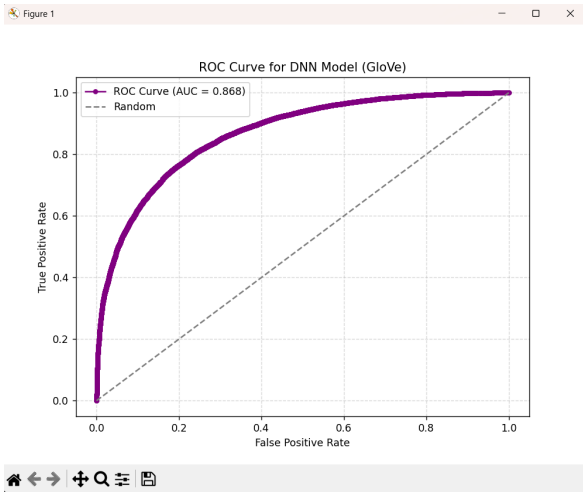
Later on, two more techniques were used - batch normalization and dropout - were added to the neural network, in order to increase its accuracy even more. Moreover, another reason was the need to decrease the slight overfitting that existed in the previous version.

In fact, with batch normalization and dropout, not only there was no overfitting but also accuracy was increased by nearly 0.7% from 0.77526 to 0.78215. Many dropout rate values were used in the range from 0.2 to 0.5. The best value of the dropout rate was 0.28.

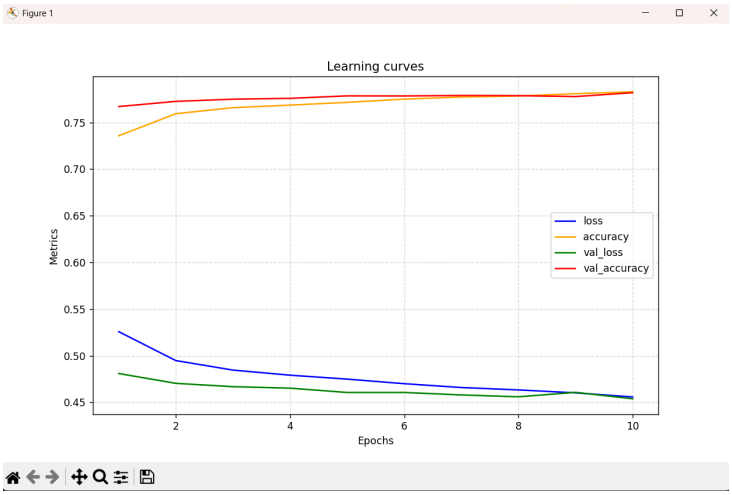
Below are the confusion matrix, the ROC curve and the learning curve for the neural network with three hidden layers, ReLU activation function, batch normalization and dropout:



(a) Confusion Matrix



(b) ROC Curve



(c) Learning Curve

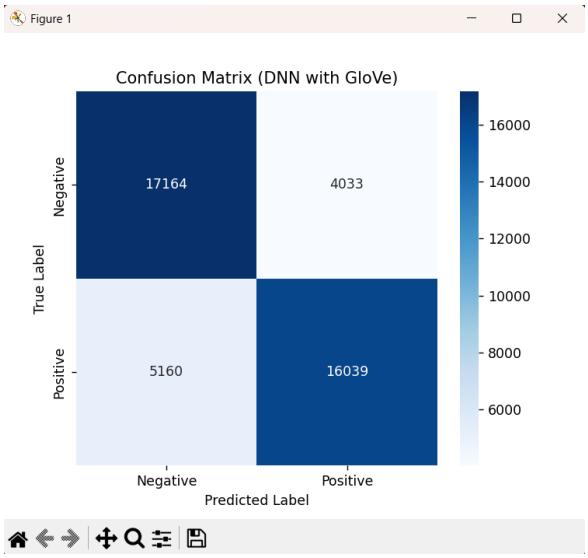
Figure 7: Performance of the neural network with three hidden layers and ReLU activation function, batch normalization and dropout

Finally, there was a need to tune hyperparameters such as the learning rate, the dropout rate and the optimizer. For this reason, 'Optuna' was considered in order to create a series of trials and to determine the best possible hyperparameters. However, ultimately 'Optuna' wasn't used and manually adjusting the hyperparameters was a better option, since monitoring the learning curves felt more important. This allowed for better understanding of how each change impacted training and validation performance and therefore it was easier to find cases of overfitting or underfitting.

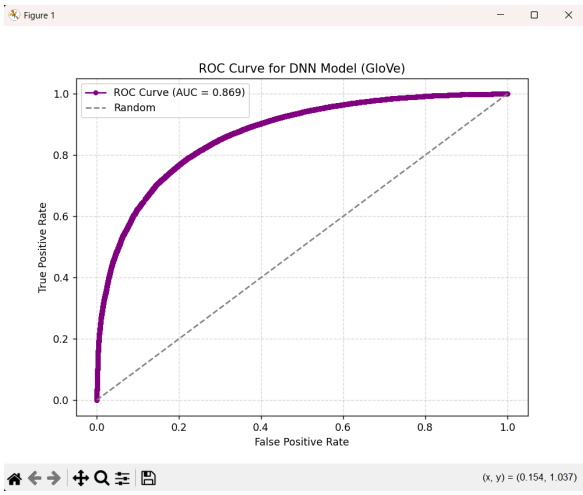
Another thing that needed tuning was the number of epochs. A golden section needed to be found in order to maintain a high accuracy while also shut down any case of overfitting.

After manual adjustments, the best optimizer was Adam, combined with a learning rate of 1e-4, a batch size of 64 and a dropout rate of 0.3. The number of epochs that was the best was 16, as it was the one with the best trade-off between accuracy and overfitting. The accuracy was 0.78316.

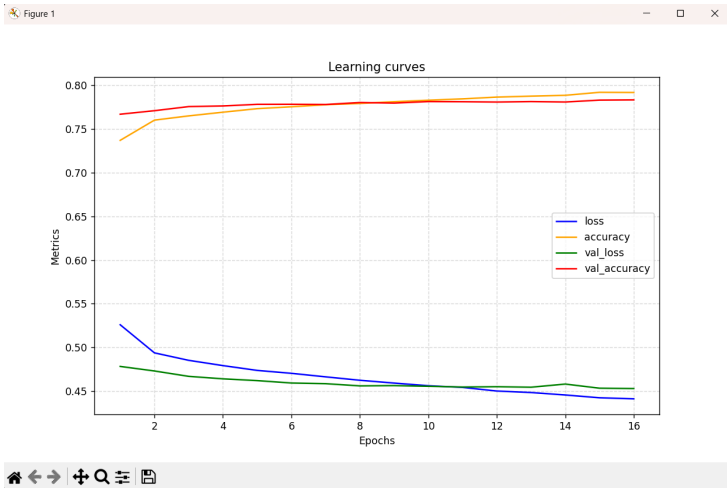
Below are the confusion matrix, the ROC curve and the learning curve for the neural network with three hidden layers, ReLU activation function, batch normalization and dropout with hyperparameters tuned:



(a) Confusion Matrix



(b) ROC Curve



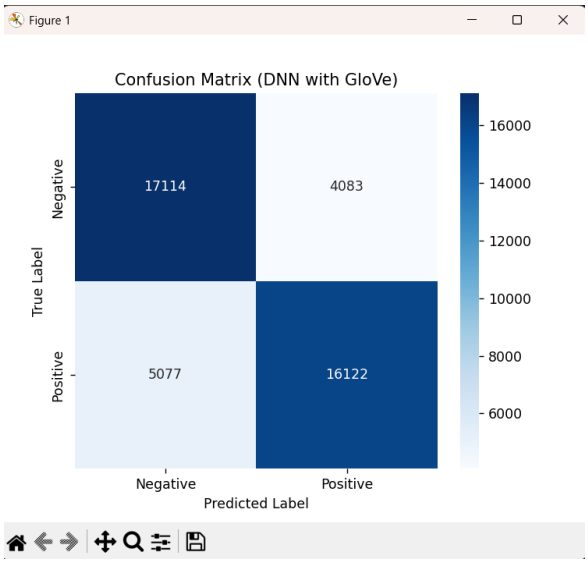
(c) Learning Curve

Figure 8: Performance of the neural network with three hidden layers and ReLU activation function, batch normalization, dropout and tuned hyperparameters

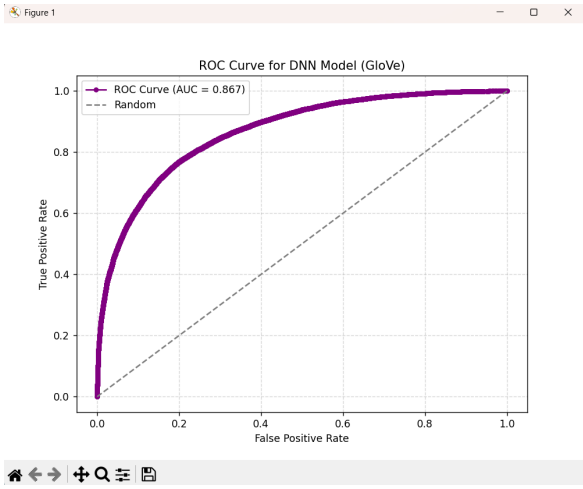
The final consideration to improve our model was adding early stopping, as well as a scheduler in order to adjust the learning rate. Since early stopping was used, the number of epochs was increased to 30, because the model would simply stop sooner if there were no improvements.

As far as the scheduler is concerned, ReduceLROnPlateau was the scheduler we tried. If the model plateus, then because of the scheduler there would be a 50% reduction of its current learning rate. The accuracy saw another increase to 0.78394

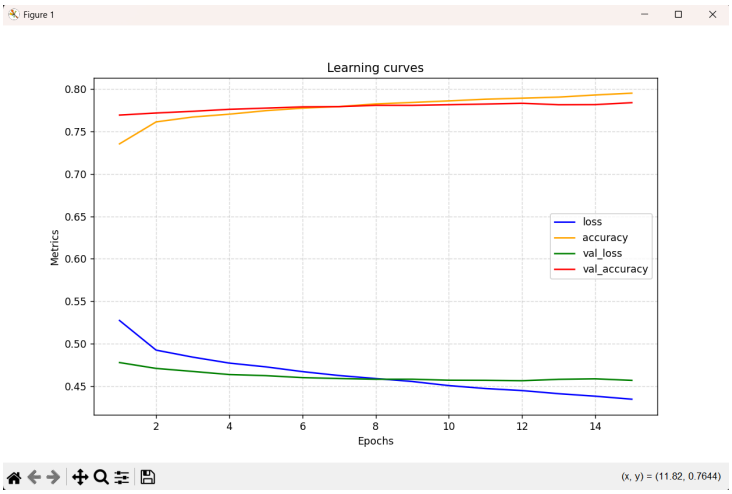
Below are the confusion matrix, the ROC curve and the learning curve for the neural network with three hidden layers, ReLU activation function, batch normalization and dropout with hyperparameters tuned. Also early stopping and a scheduler are now used:



(a) Confusion Matrix



(b) ROC Curve



(c) Learning Curve

Figure 9: Performance of the neural network with 3 hidden layers, ReLU, batch normalization, dropout, scheduler, early stopping and tuned hyperparameters

3.1.1. Table of trials. Below we show the table of trials that were done in order to improve our model. A synopsis of the trial is presented alongside the accuracy score for each model. The main idea was to start with a very simple neural network (only two hidden layers), and later to increase complexity gradually.

Our goal was successful since the final model contains three hidden layers, an activation function, batch normalization, dropout, hyperparamaters (learning rate, batch size, optimizer and dropout rate) tuned, a scheduler and also uses the early stopping technique.

As the complexity of the model increased, the increase in accuracy was noticable as it jumped almost 3%, from 0.75455 which was the accuracy of the very simple model, to 0.78394 which was the accuracy of the much more complex and efficient model of our neural network.

Below is the Table of trials for our neural network model.

Trial	Score
2 hidden layer neural network	0.75455
3 hidden layer neural network	0.75653
Adding ReLU activation function	0.77526
Adding batch normalization and dropout	0.78215
Hyperparameter tuning	0.78316
Adding scheduler and early stopping	0.78394

Table 1: Summary of trials done to improve accuracy on the model

3.2. Hyper-parameter tuning

To better tune the model, the hyperparameters needed to be tuned At first the option to use tools like 'Optuna' seemed like the obvious one, in order to find the best hyper parameters for the model. However, it was eventually not chosen as it did not give direct feedback and insight on how well the model generalized because the learning curves weren't seen during each attempt. So, manual tuning was instead preferred which led to a better understanding of the trade-off between the hyperparameters and the generalization of the model.

3.3. Optimization techniques

To optimize our model apart from hyper-parameter tuning the following were used:

1. Early Stopping: Early stopping was used in order to stop training the model if it did not improve its results over a few epochs. This helped avoid overfitting and also gave the model the ability to reach its best performance

2. ReLU Activation Function: The activation function that was used was 'ReLU'. Due to its simplicity, ReLU provided the best possible results, while also achieving a huge increase in accuracy due to its non-linearity.

3. Batch normalization and Dropout Adding both dropout and batch normalization led to a more stable and regularized training process, leading to higher accuracy and reduction of overfitting.

4. Learning Rate Scheduler: Finally, adding a learning rate scheduler helped to change the learning rate each time the validation loss plateaued. When the validation loss plateaued, the learning rate was reduced by 0.5. With a scheduler, weight updates are done smoother and convergence is better.

All these techniques, alongside a strong neural network architecture and hyperparameter tuning, lead to an optimizer model with a good ability to discriminate between positive and negative sentiments, and with an accuracy of about 0.78394

3.4. Evaluation

To evaluate the performance of the model, the evaluation metric that is used is accuracy. Since our dataset is balanced, and there is no priority of false negatives over false positives (or vice versa) as both types of misclassification have similar consequences, accuracy is the preferred metric.

The scores of recall, precision and F1-score are given below:

```
DNN Model Evaluation:
Accuracy: 0.78394
Precision: 0.79792
Recall: 0.76051
F1 Score: 0.77877
              precision    recall  f1-score   support

           0       0.77      0.81      0.79     21197
           1       0.80      0.76      0.78     21199

    accuracy                           0.78     42396
   macro avg       0.78      0.78      0.78     42396
weighted avg       0.78      0.78      0.78     42396
```

Figure 10: Different evaluation metrics for the final model

The above results show the precision-recall trade-off, which was mentioned earlier. The F1-score is a balance between these two metrics. Since the dataset is balanced, accuracy is also close to the F1-score, reflecting the overall performance without being skewed by class imbalance.

3.4.1. ROC curve. Another useful way to evalute our model is through the ROC curve. The receiver operating characteristic (ROC) curve is a tool used for evaluating the performance of a binary classifier. The ROC curve plots the true positive rate against the false positive rate at different thresholds

The ROC curve of the final model is given below

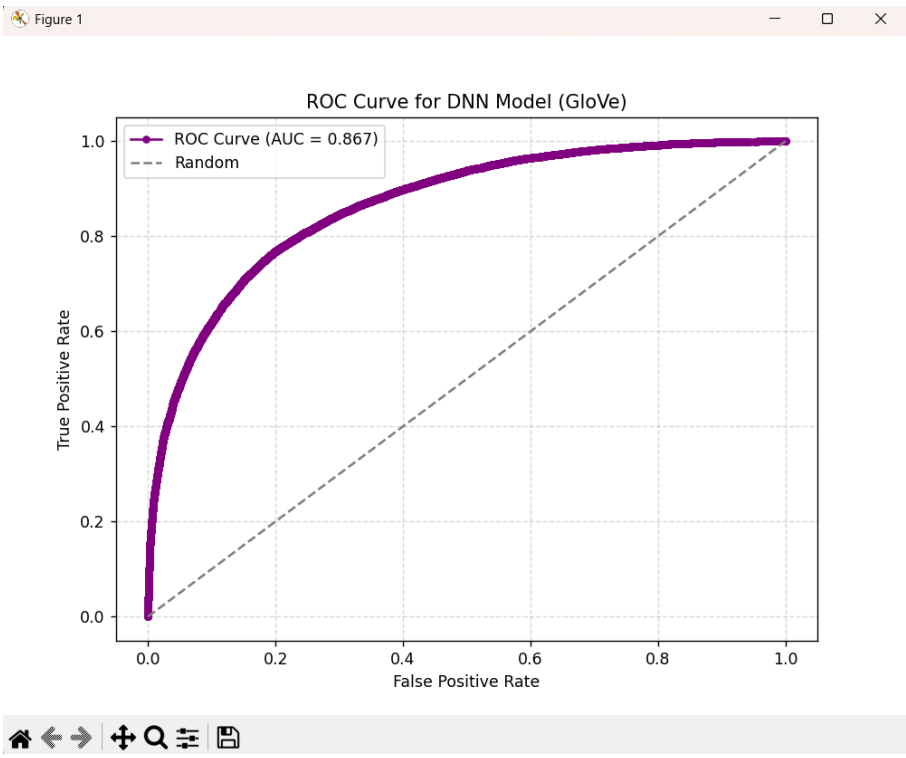


Figure 11: ROC curve for the final model

The ROC curve has an Area Under Curve (AUC) of 0.867 which shows a good ability to discriminate between positive and negative sentiments, compared to a random classifier.

3.4.2. Learning Curve. The learning curves are another good way to evaluate the final model. A learning curve shows the performance of the model against training size. Four curves are shown simultaneously. Two curves that concern the training performance(training accuracy and training loss) and two curves that concern the validation performance(validation accuracy and validation loss).

Below are the learning curves of the final model.

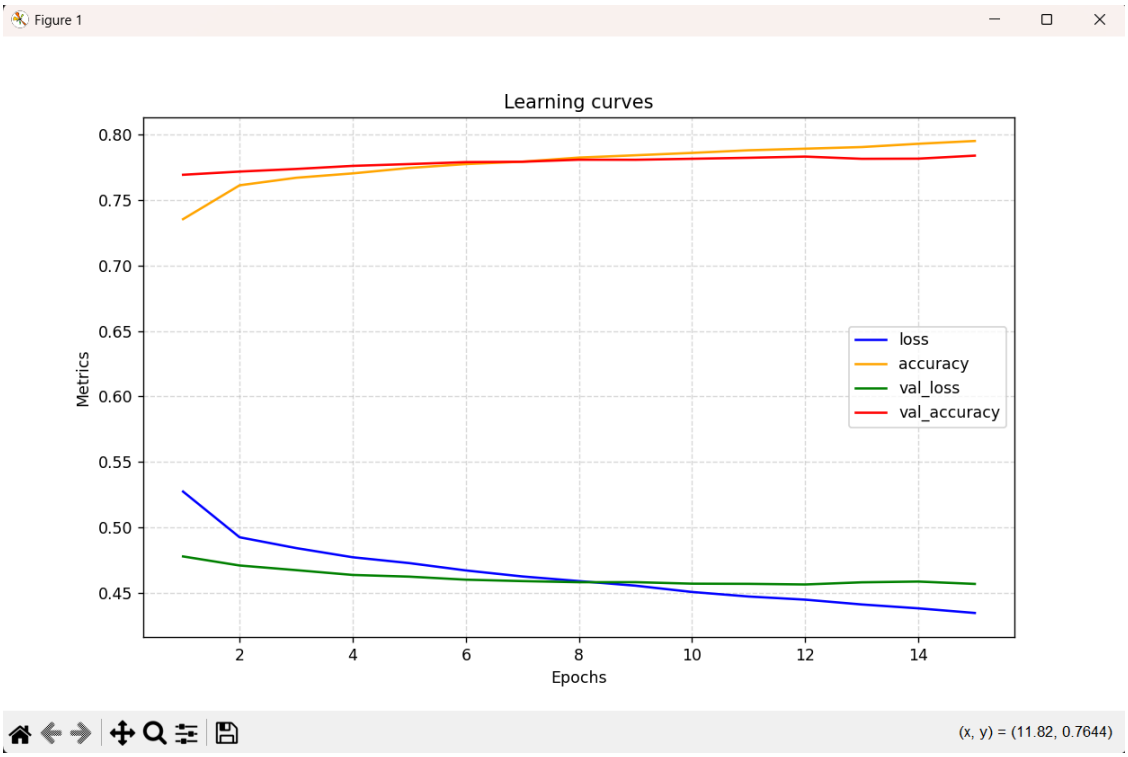


Figure 12: Learning curves for the final model

From the learning curves, is is clear that there is minimal overfitting, as the training and validation accuracy curves remain close to each other through the passing of epochs. Both accuracy curves show an upward trend. Similarly, the training and validation loss curves follow a downward trend and show signs of flattening, further proving that the model is not overfitting a lot. Considering these trends, the model reflects a good trade-off between model performance and generalization capability.

3.4.3. Confusion matrix. Finally, a better way to evaluate the performance of our model is the confusion matrix. The confusion matrix works like this: Each row of a confusion matrix represents an actual class, while each column represents a predicted class. Therefore, true positives, true negatives, false positives and false negatives all have a numerical value in the confusion matrix. Therefore, a confusion matrix gives a vizual representation of metrics beyond accuracy such as recall and precision and is useful in identifying possible problems with our model.

Below is the confusion matrix of the final model.

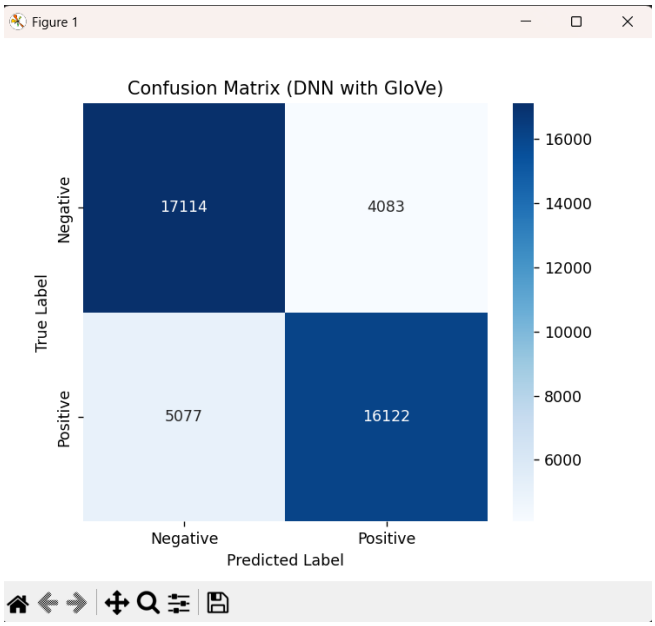


Figure 13: Confusion matrix for the final model

The confusion matrix shows that the number of False Positives is lower than the number of False Negatives. As a result, the recall score is lower than the precision score. However, since the True Positives and True Negatives are significantly higher overall, the model demonstrates high accuracy.

4. Results and Overall Analysis

4.1. Results Analysis

The final model scores an accuracy of 78.394%, while the generalization gap between the training and the validation data set is less than 2%. This is considered a fair result and a good trade-off between accuracy score and possible overfitting of the model.

One of the most impactful changes to our neural network was the addition of a ReLU activation function. Introducing non-linearity significantly improved the model's performance, leading to an accuracy increase of nearly 2%. This shows the importance of having network which is being able to capture complex patterns and relationships in the data, rather than being limited to only linear connections. More complex activation functions such as Leaky ReLU, SELU, and GELU were tested but did not outperform ReLU, likely due to the relatively simple nature of the classification task.

The learning curve and ROC curve show that the classifier is much better at separating positive from negative sentiments on tweets compared to a random one. At the same time, the overfitting of less than 2% is a good balance between the accuracy of the model and its ability to generalize well.

Furthermore, it would be ideal for all wrong spellings and slang would be replaced with the correct words, in order for less noise to occur in the training data. However it is essentially impossible to include in the pre processing function every possible wrong spelling or every possible slang that would be used in a tweet. Therefore, only common misspellings and common slang was replaced.

In conclusion, the gradual change of the model, from having a simple initial archi-

tecture to having a more complex one with activation functions, batch normalization, dropout and hyperparameter tuning led to much better results and we ended up with a robust final neural network.

4.1.1. Best trial. The model's best trial achieved accuracy of 78.394% with overfitting of less than 2%. This was done using 3 hidden layers with 512, 256 and 64 neurons in each hidden layer respectively. Also ReLU activation function was used with no weight initialization. As far as the optimizer is concerned 'Adam' was chosen with a learning rate of '1e-4'. Furthermore, scheduler 'ReduceLROnPlateau' was used to change the learning rate each time the validation loss plateaued. Finally, a batch size of 64 was used.

The techniques of batch normalization, dropout and early stopping were also handy and led to a more robust model with higher accuracy and lower generalization gap.

Moreover, a big part in making the model effective was the pre-processing function which included converting every letter to lowercase, correcting common spelling mistakes and converting 'slang' terms to their actual meaning. Additionally, removing URLs, mentions, hashtags, non-ASCII characters and punctuation (except for exclamation marks and question marks) increased accuracy. Converting more than 3 characters into one (e.g. converting 'heyyyy' into 'hey') was beneficial as it reduced noise and cleaned the data.

4.2. Comparison with the first project

The first and second project had very similar results with about 2% percent difference. Accuracy on the first project with the Linear classifier was around 80.373%, while accuracy on the second project with the neural network was at around 78.394%. This is likely due to the fact that a neural network is much more complex than logistic regression. Additionally the nature of tweets is noisy, so a neural network might be learning more complex patterns because of noise in the tweets. Therefore, it is logical to have a slightly worse result because of the simple nature of our task.

5. Bibliography

References

[1] Classification: Roc and auc, 2024. https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc.

[2] Manolis Koubarakis. Artificial intelligence ii course homepage for the department of informatics and telecommunications, 2025. https://cgi.di.uoa.gr/~ys19/.

[3] Manolis Koubarakis. Backpropagation, 2025. https://cgi.di.uoa.gr/~ys19/lectures25/backpropagation.pdf.

[4] Manolis Koubarakis. Perceptrons, 2025. https://cgi.di.uoa.gr/~ys19/lectures25/perceptrons.pdf.

[5] Manolis Koubarakis. Training deep neural networks, 2025. https://cgi.di.uoa.gr/~ys19/lectures25/training_DNNs.pdf.

[6] Manolis Koubarakis. Word vectors, 2025. https://cgi.di.uoa.gr/~ys19/lectures25/word_vectors.pdf.

[7] Manolis Koubarakis. Word2vec, 2025. https://cgi.di.uoa.gr/~ys19/lectures25/word2vec.pdf.

You can refer to the material from [1], [2], [3], [4], [5], [6] and [7].